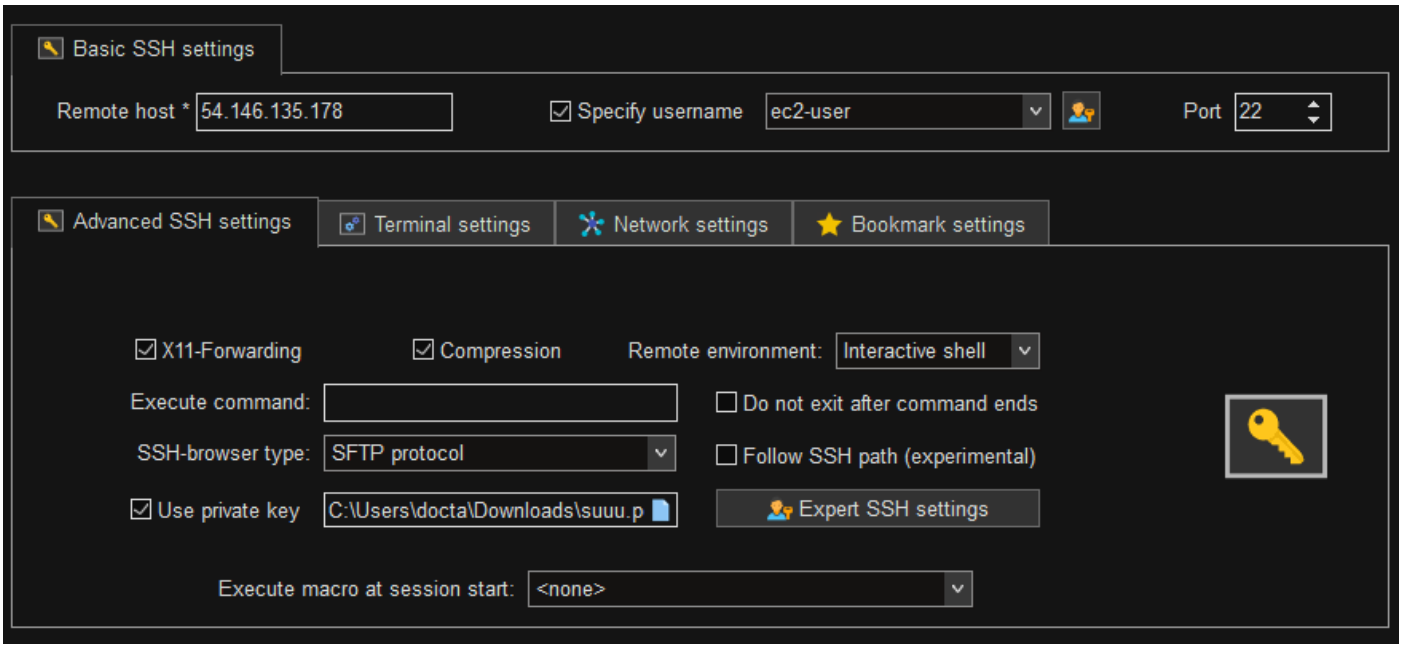
Ejercicio 1: Instalación de Docker

1. **Conéctate a tu instancia EC2 mediante SSH.**



Hacemos conexión mediante SSH a la maquina EC2 que hemos creado agregamos los datos necesarios IP publica, username y el archivo vocky. En mi caso para hacer la conexión uso MobaXterm ya que se me hace más cómodo que otros clientes SSH.

2. **Actualiza los paquetes del sistema:**

```
[ec2-user@ip-172-31-92-203 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-92-203 ~]$
```

Usamos los comandos de la consola Linux (sudo yum update -y) para actualizar los paquetes de la maquina EC2 como la maquina esta recién instalada vemos que no instala nada ya que esta todo actualizado.

3. **Instala Docker:**

```
[ec2-user@ip-172-31-92-203 ~]$ sudo yum install docker -y
çLast metadata expiration check: 0:05:41 ago on Mon Mar  3 19:05:48 2025.
Dependencies resolved.
================================================================================
 Package                 Architecture  Version                    Repository     Size
================================================================================
Installing:
 docker                  x86_64        25.0.8-1.amzn2023.0.1      amazonlinux    44 M
Installing dependencies:
 containerd              x86_64        1.7.25-1.amzn2023.0.1      amazonlinux    36 M
 iptables-libs           x86_64        1.8.8-3.amzn2023.0.2       amazonlinux   401 k
 iptables-nft            x86_64        1.8.8-3.amzn2023.0.2       amazonlinux   183 k
 libcgroup               x86_64        3.0-1.amzn2023.0.1         amazonlinux    75 k
 libnetfilter_conntrack  x86_64        1.0.8-2.amzn2023.0.2       amazonlinux    58 k
 libnfnetlink            x86_64        1.0.1-19.amzn2023.0.2      amazonlinux    30 k
 libnftnl                x86_64        1.2.2-2.amzn2023.0.2       amazonlinux    84 k
 pigz                    x86_64        2.5-1.amzn2023.0.3         amazonlinux    83 k
 runc                    x86_64        1.2.4-1.amzn2023.0.1       amazonlinux   3.4 M
```

Una vez tengamos actualizada la maquina a su última versión ya podemos bajarnos la última versión de Docker mediante el siguiente comando (sudo yum install docker -y) vemos como ya ha instalado todos los paquetes necesarios para Docker.

4. **Inicia el servicio de Docker:**

```
[ec2-user@ip-172-31-92-203 ~]$ sudo systemctl start docker
```

Usamos (sudo systemctl start docker) para iniciar el programa de Docker y poder a empezar a trabajar con el.

5. **Habilita Docker para que se inicie automáticamente al reiniciar:**

```
[ec2-user@ip-172-31-92-203 ~]$ sudo systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
```

Ahora seguimos usando este comando (sudo systemctl enable docker) para automatizar el Docker y hacer que se inicie automáticamente al reiniciar al instancia EC2.

6. **Verifica la instalación:**

```
[ec2-user@ip-172-31-92-203 ~]$ docker --version
Docker version 25.0.8, build 0bab007
```

Verificamos que la instalación se ha hecho bien usando (docker --version) y así ver la versión de Docker.

Ejercicio 2: Ejecutar tu primer contenedor

1. **Ejecuta tu primer contenedor de prueba:**

```
[ec2-user@ip-172-31-92-203 ~]$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
e6590344b1a5: Pull complete
Digest: sha256:bfbb0cc14f13f9ed1ae86abc2b9f11181dc50d779807ed3a3c5e55a6936dbdd5
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

[ec2-user@ip-172-31-92-203 ~]$
```

El contenedor hello-world es una imagen oficial de Docker que se utiliza para comprobar que la instalación de Docker funciona correctamente. Cuando se ejecuta el comando, el cliente Docker se conecta al daemon, que es el servicio que gestiona los contenedores en el sistema.

Si la imagen no existe localmente, Docker la descarga desde Docker Hub y crea un contenedor a partir de ella. El contenedor se ejecuta y muestra un mensaje explicando el proceso de comunicación entre el cliente Docker, el daemon y el contenedor.

Este contenedor es muy sencillo, su única función es imprimir ese mensaje y luego terminar su ejecución. Sirve como primer paso para familiarizarse con Docker y comprobar que todo está configurado correctamente.

Ejercicio 3: Descargar y ejecutar una imagen de Nginx

1. **Descarga la imagen de Nginx desde Docker Hub:**

```
[ec2-user@ip-172-31-92-203 ~]$ sudo docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
7cf63256a31a: Pull complete
bf9acace214a: Pull complete
513c3649bb14: Pull complete
d014f92d532d: Pull complete
9dd21ad5a4a6: Pull complete
943ea0f0c2e4: Pull complete
103f50cb3e9f: Pull complete
Digest: sha256:9d6b58feebd2dbd3c56ab5853333d627cc6e281011cfd6050fa4bcf2072c9496
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Usamos el comando (sudo docker pull nginx) para ir a docker hub y desde allí descargar la imagen de nginx e instalarlo en nuestro Docker y así no tener que instalarlo manualmente.

2. **Ejecuta un contenedor con Nginx en el puerto 8080:**

```
[ec2-user@ip-172-31-92-203 ~]$ sudo docker run -d -p 8080:80 nginx
46c7f6effaadf1b8598526a56b99bbf32c17460f8d67e1d037cac1a3fe653b29
```

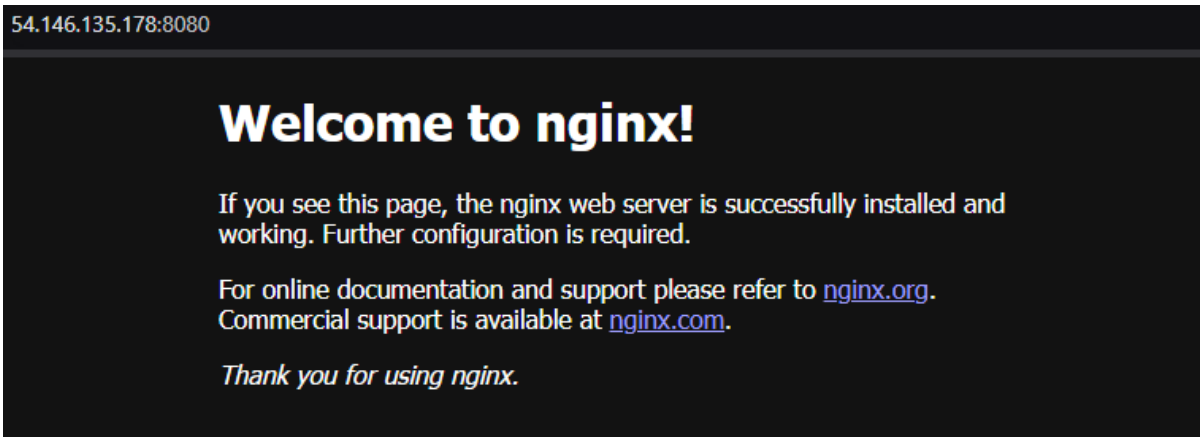
Con (sudo docker run -d -p 8080:80 nginx) ejecutamos un contenedor de Docker con nginx al cual nos conectaremos mediante el puerto 8080. Debemos asegurarnos de que hemos habilitado dicho puerto en el Security Group del EC2.

3. **Verifica que el contenedor está corriendo:**

```
[ec2-user@ip-172-31-92-203 ~]$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                   NAMES
46c7f6effaad   nginx     "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   0.0.0.0:8080->80/tcp, :::8080->80/tcp   clever_fermi
```

Usando (docker ps) vemos la información de nuestro Docker y en efecto se ha creado correctamente el contenedor con nginx en el puerto 8080.

4. **Abre un navegador y accede a http:// 54.146.135.178:8080 para comprobar que Nginx está funcionando:**



Y vemos que si nos vamos al nuestro navegador y pegamos http://54.146.135.178:8080 nos lleva a nuestra pagina de nginx donde nos saldrá el mensaje por defecto.

Ejercicio 4: Inspeccionar un contenedor en ejecución

1. **Lista todos los contenedores en ejecución:**

```
[ec2-user@ip-172-31-92-203 ~]$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                     NAMES
46c7f6effaad   nginx     "/docker-entrypoint.…"   3 minutes ago   Up 3 minutes   0.0.0.0:8080->80/tcp, :::8080->80/tcp     clever_fermi
```

Usamos el mismo comando anterior para ver la lista de todos los contenedores (docker ps) y vemos que sigue en ejecución nuestro anterior contenedor.

2. **Obtén información detallada de un contenedor:**


```
[ec2-user@ip-172-31-92-203 ~]$ sudo docker inspect 46c7f6effaad
[
    {
        "Id": "46c7f6effaadf1b8598526a56b99bbf32c17460f8d67e1d037cac1a3fe653b29",
        "Created": "2025-03-03T19:52:56.168722744Z",
        "Path": "/docker-entrypoint.sh",
        "Args": [
            "nginx",
            "-g",
            "daemon off;"
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 29541,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2025-03-03T19:52:56.636907602Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
        },
        "Image": "sha256:b52e0b094bc0e26c9eddc9e4ab7a64ce0033c3360d8b7ad4ff4132c4e03e8f7b",
        "ResolvConfPath": "/var/lib/docker/containers/46c7f6effaadf1b8598526a56b99bbf32c17460f8d67e1d037cac1a3fe653b29/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/46c7f6effaadf1b8598526a56b99bbf32c17460f8d67e1d037cac1a3fe653b29/hostname",
        "HostsPath": "/var/lib/docker/containers/46c7f6effaadf1b8598526a56b99bbf32c17460f8d67e1d037cac1a3fe653b29/hosts",
        "LogPath": "/var/lib/docker/containers/46c7f6effaadf1b8598526a56b99bbf32c17460f8d67e1d037cac1a3fe653b29/46c7f6effaadf1b8598526a56b99bbf32c17460f8d67e1d037cac1a3fe653b29-json.log",
        "Name": "/clever_fermi",
        "RestartCount": 0,
        "Driver": "overlay2",
        "Platform": "linux",
        "MountLabel": "",
        "ProcessLabel": "",
        "AppArmorProfile": "",
        "ExecIDs": null,
        "HostConfig": {
            "Binds": null,
            "ContainerIDFile": "",
            "LogConfig": {
                "Type": "json-file",
                "Config": {}
            },
            "NetworkMode": "default",
            "PortBindings": {
                "80/tcp": [
                    {
                        "HostIp": "",
                        "HostPort": "8080"
                    }
                ]
            },
            "RestartPolicy": {
                "Name": "no",
                "MaximumRetryCount": 0
            },
            "AutoRemove": false,
```


Ahora vamos a revisar el contenedor mas a fondo con este comando (sudo docker inspect 46c7f6effaad) donde especificamos el id de la instancia de Docker que queremos ver en profundidad.

3. **Explica qué datos muestra el comando docker inspect.**

El comando docker inspect muestra información detallada sobre un contenedor o una imagen de Docker. Al ejecutarlo, devuelve un bloque de datos en formato JSON con diferentes propiedades relacionadas con el objeto de que se inspecciona. Estos datos incluyen información sobre su configuración, estado y funcionamiento.

Algunas de las propiedades más importantes son el ID, el nombre del contenedor, la imagen que utiliza y la fecha de creación. También muestra detalles sobre la configuración de red, como la dirección IP asignada, los puertos expuestos y cómo están mapeados a puertos en el host.

Además, incluye información sobre los volúmenes utilizados, especificando si hay carpetas compartidas entre el contenedor y el sistema anfitrión. También muestra las variables de entorno configuradas dentro del contenedor, lo que es útil para entender cómo está preparado para su ejecución.

Ejercicio 5: Crear un contenedor interactivo con Ubuntu

1. **Ejecuta un contenedor Ubuntu en modo interactivo:**

```
[ec2-user@ip-172-31-92-203 ~]$ sudo docker run -it ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
5a7813e071bf: Pull complete
Digest: sha256:72297848456d5d37d1262630108ab308d3e9ec7ed1c3286a32fe09856619a782
Status: Downloaded newer image for ubuntu:latest
```

Usamos el comando (sudo docker run -it ubuntu bash) para ejecutar un contenedor con una imagen de Ubuntu que este en modo Interactivo.

2. **Ejecuta algunos comandos dentro del contenedor:**

```
root@bfc83d7ec299:/# ls -l
total 0
lrwxrwxrwx.   1 root root   7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x.   2 root root   6 Apr 22  2024 boot
drwxr-xr-x.   5 root root 360 Mar  3 20:39 dev
drwxr-xr-x.   1 root root  66 Mar  3 20:39 etc
drwxr-xr-x.   3 root root  20 Jan 27 02:09 home
lrwxrwxrwx.   1 root root   7 Apr 22  2024 lib -> usr/lib
lrwxrwxrwx.   1 root root   9 Apr 22  2024 lib64 -> usr/lib64
drwxr-xr-x.   2 root root   6 Jan 27 02:03 media
drwxr-xr-x.   2 root root   6 Jan 27 02:03 mnt
drwxr-xr-x.   2 root root   6 Jan 27 02:03 opt
dr-xr-xr-x. 172 root root   0 Mar  3 20:39 proc
drwx------.   2 root root  37 Jan 27 02:09 root
drwxr-xr-x.   4 root root  33 Jan 27 02:09 run
lrwxrwxrwx.   1 root root   8 Apr 22  2024 sbin -> usr/sbin
drwxr-xr-x.   2 root root   6 Jan 27 02:03 srv
dr-xr-xr-x.  13 root root   0 Mar  3 18:49 sys
drwxrwxrwt.   2 root root   6 Jan 27 02:09 tmp
drwxr-xr-x.  12 root root 133 Jan 27 02:03 usr
drwxr-xr-x.  11 root root 139 Jan 27 02:09 var
root@bfc83d7ec299:/#
```

Una vez dentro de la imagen de Ubunto ejecutamos algún comando para ver que funciona correctamente como por ejemplo (ls -l).

3. **Sal del contenedor:**

```
root@bfc83d7ec299:/# exit
exit
[ec2-user@ip-172-31-92-203 ~]$
```

Y con el comando (exit) podemos salir de la maquina de ubunto y volver a nuestra maquina EC2

Ejercicio 6: Construir una imagen personalizada

1. **Crea un directorio para el proyecto:**

```
[ec2-user@ip-172-31-92-203 ~]$ mkdir docker-test && cd docker-test
[ec2-user@ip-172-31-92-203 docker-test]$
```

Creamos un nuevo directorio para nuestro proyecto (mkdir docker-test && cd docker-test) y nos posicionamos en el directorio que acabamos de crear.

2. **Crea un archivo Dockerfile y añade lo siguiente:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo nano Dockerfile
```

```
  GNU nano 5.8
FROM ubuntu
RUN apt update && apt install -y curl
CMD ["bash"]
```

Nos creamos el archivo Dockerfile con nano (sudo nano Dockerfile) y en el escribimos los datos del docker file necesarios y lo guardamos

3. **Construye la imagen:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker build -t mi-imagen .
[+] Building 10.2s (6/6) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 159B
 => [internal] load metadata for docker.io/library/ubuntu:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/2] FROM docker.io/library/ubuntu:latest
 => [2/2] RUN apt update && apt install -y curl
 => exporting to image
 => => exporting layers
 => => writing image sha256:7480d432035c826dbc2ae4053de2bedcb2a4ffcf1c3ade46b9e162fa6e5a590c
 => => naming to docker.io/library/mi-imagen
```

Usamos el comando (sudo docker build -t mi-imagen .) para construir nuestra imagen de docker.

4. **Ejecuta un contenedor con la imagen:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker run -it mi-imagen
root@555ce2c14da5:/#
```

ahora ejecutamos nuestra imagen de docker (sudo docker run -it mi-imagen) y vemos que se ha creado correctamente tal y como la hemos configurado.

Ejercicio 7: Configurar variables de entorno en un contenedor

1. **Ejecuta un contenedor con una variable de entorno personalizada:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker run -e "MI_VARIABLE=HolaMundo" ubuntu env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=f8739b05a837
MI_VARIABLE=HolaMundo
HOME=/root
```

Ejecutamos un contenedor, pero con una variable de entorno personalizada para este caso con el comando (sudo docker run -e "MI_VARIABLE=HolaMundo" ubuntu env)

2. **Explica qué hace el comando anterior y qué salida genera:**

Este comando crea y ejecuta un contenedor basado en la imagen de Ubuntu. La opción -e sirve para definir una variable de entorno llamada MI_VARIABLE con el valor "HolaMundo", que estará disponible dentro del contenedor mientras se ejecuta. El comando env es el que se ejecuta dentro del contenedor, y muestra todas las variables de entorno disponibles.

La salida generada es una lista de variables de entorno dentro del contenedor, donde se puede ver MI_VARIABLE=HolaMundo, lo cual confirma que la variable se pasó correctamente. También aparecen otras variables típicas como PATH, HOSTNAME y HOME, que son propias del sistema dentro del contenedor.

Ejercicio 8: Montar volúmenes para persistencia de datos

1. **Crea un volumen:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker volume create mi-volumen
mi-volumen
```

Usamos el comando (sudo docker volume create mi-volumen) para crearnos un volumen propio.

2. **Ejecuta un contenedor con el volumen montado:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker run -v mi-volumen:/data ubuntu bash
[ec2-user@ip-172-31-92-203 docker-test]$
```

Ahora simplemente ejecutamos el contenedor, pero añadiendo nuestro volumen con este comando (sudo docker run -v mi-volumen:/data ubuntu bash)

3. **Crea un archivo dentro del contenedor y verifica que persiste:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo sudo docker run -it -v mi-volumen:/data ubuntu bash
root@cea6bfbc5878:/# echo "Mi archibo de prueba para ver si se mantiene" > /data/archivo.txt
root@cea6bfbc5878:/# exit
exit
```

Entramos en nuestro contenedor con (sudo docker run -it -v mi-volumen:/data ubuntu bash) y creamos un archivo de prueba (echo "Mi archibo de prueba para ver si se mantiene" > /data/archivo.txt) y salimos de nuestro contenedor.

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker run -it -v mi-volumen:/data ubuntu bash
root@d379b66b25ca:/# cat /data/archivo.txt
Mi archibo de prueba para ver si se mantiene
root@d379b66b25ca:/#
```

Y al levantar un nuevo contenedor con nuestro volumen vemos que se mantiene el archivo que hemos creado.

Ejercicio 9: Crear y administrar redes en Docker

1. **Crea una red personalizada:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker network create mi-red
6cb162482e22a412fe13a7bdb544b1885c953a9bd409a83c6b671f64dbac3df8
```

Creamos nuestra red personalizada con este comando (sudo docker network create mi-red) y al llamamos mi -red

2. **Conecta un contenedor a la red:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker run -dit --network mi-red --name contenedor1 ubuntu
522f1202b8edd85cc8199ebf6b0258148af86c60ad72447df4d6709e0fba4abb
```

Ahora con esto (sudo docker run -dit --network mi-red --name contenedor1 ubuntu) conectamos nuestro contenedor a nuestra red personalizada.

3. **Verifica la conexión entre contenedores usando ping:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' contenedor1
172.18.0.2
```

Lo primero será buscar la ip de nuestra red personalizada con (sudo docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' contenedor1)

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker run -dit --network mi-red --name contenedor5 ubuntu
b47c15eeb829b67c173adc1233e691367d550ba8a50043be47e98765536267a8
```

Nos creamos un nuevo contenedor con el comando (sudo docker run -dit --network mi-red --name contenedor5 ubuntu)

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker exec -it contenedor5 bash
root@b47c15eeb829:/#
```

Accedemos a nuestro nuevo contenedor que hemos creado con (sudo docker exec -it contenedor5 bash)

```
root@b47c15eeb829:/# ping 172.18.0.2
PING 172.18.0.2 (172.18.0.2) 56(84) bytes of data.
64 bytes from 172.18.0.2: icmp_seq=1 ttl=127 time=0.068 ms
64 bytes from 172.18.0.2: icmp_seq=2 ttl=127 time=0.056 ms
64 bytes from 172.18.0.2: icmp_seq=3 ttl=127 time=0.063 ms
64 bytes from 172.18.0.2: icmp_seq=4 ttl=127 time=0.054 ms
^C
--- 172.18.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3142ms
rtt min/avg/max/mdev = 0.054/0.060/0.068/0.005 ms
root@b47c15eeb829:/#
```

Y ahora si hacemos ping a la IP que nos ha dado antes vemos como sí que tenemos conexión ping.

Ejercicio 10: Ejecutar una base de datos en Docker

1. **Ejecuta un contenedor MySQL:**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker run --name mysql-db -e MYSQL_ROOT_PASSWORD=1234 -d mysql
Unable to find image 'mysql:latest' locally
latest: Pulling from library/mysql
43759093d4f6: Pull complete
d255dceb9ed5: Pull complete
23d22e42ea50: Pull complete
431b106548a3: Pull complete
2be0d473cadf: Pull complete
f56a22f949f9: Pull complete
277ab5f6ddde: Pull complete
df1ba1ac457a: Pull complete
cc9646b08259: Pull complete
893b018337e2: Pull complete
Digest: sha256:146682692a3aa409eae7b7dc6a30f637c6cb49b6ca901c2cd160becc81127d3b
Status: Downloaded newer image for mysql:latest
7636e518274eb7dde40abd00da79b7e430f5b95f2d0fa0a8f7b27bad04dff8af
```

Ejecutamos un contenedor y en el vamos a instalar una base de datos My SQL con este comando (sudo docker run --name mysql-db -e MYSQL_ROOT_PASSWORD=1234 -d mysql)

2. **Conéctate a la base de datos dentro del contenedor.**

```
[ec2-user@ip-172-31-92-203 docker-test]$ sudo docker exec -it mysql-db bash
bash-5.1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 9.2.0 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Accedemos al contenedor con (sudo docker exec -it mysql-db bash) y dentro accedemos a la base de datos con (mysql -u root -p) y vemos como ha funcionado exitosamente la instalación de My SQL en el contenedor.